



中山大學
SUN YAT-SEN UNIVERSITY

C++语言标准模板库(STL)

中山大学计算机类专业教程



主讲人：万海



wanhai@mail.sysu.edu.cn

目录

CONTENTS

01

STL是什么

02

容器

2.1

顺序容器

2.2

关联容器

2.3

适配器

03

迭代器

04

算法

05

其他（选讲）



容器适配器

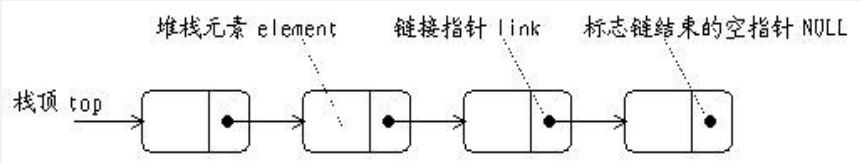
- 容器适配器提供顺序容器之上的不同功能接口（界面）。

类 名	说 明	所在头文件
stack	栈。元素的插入和删除只在栈顶进行，支持后进先出（LIFO）的元素访问	<stack>
queue	队列。在尾端插入元素，在头端删除元素，支持先进先出（FIFO）的元素访问	<queue>
priority_deque	带优先级管理的队列。元素按优先级从高到低排列，使用元素类型的<操作确定优先级	<queue>



栈的功能接口（操作）

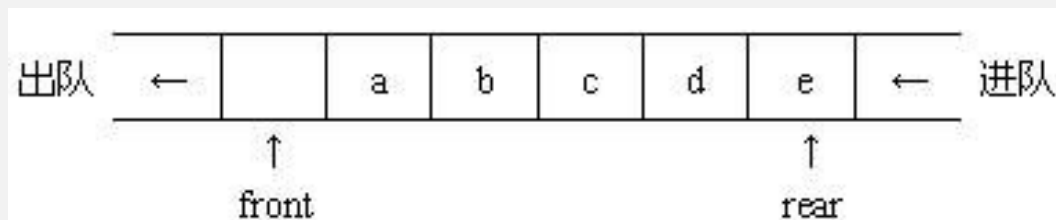
操 作	操作效果	实现方式
push(item)	将值为item的新元素压入栈顶，无返回值	调用基础容器的push_back操作实现
pop()	删除栈顶元素，无返回值	调用基础容器的pop_back操作实现
top()	返回栈顶元素的值	调用基础容器的back操作实现
empty()	若栈为空，则返回true；否则，返回false	调用基础容器的empty操作实现
size()	返回栈中元素的数目	调用基础容器的size操作实现





队列的功能接口（操作）

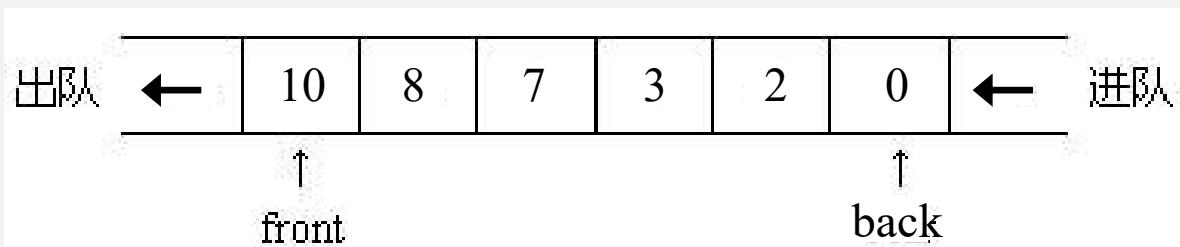
操 作	操作效果	实现方式
<code>push(item)</code>	将值为 <code>item</code> 的新元素插入队尾，无返回值	调用基础容器的 <code>push_back</code> 操作实现
<code>pop()</code>	删除队首元素，无返回值	调用基础容器的 <code>pop_front</code> 操作实现
<code>front()</code>	返回队首元素的值	调用基础容器的 <code>front</code> 操作实现
<code>back()</code>	返回队尾元素的值	调用基础容器的 <code>back</code> 操作实现
<code>empty()</code>	若队列为空，则返回 <code>true</code> ；否则，返回 <code>false</code>	调用基础容器的 <code>empty</code> 操作实现
<code>size()</code>	返回队列中元素的数目	调用基础容器的 <code>size</code> 操作实现





优先队列的功能接口（操作）

操 作	操作效果	实现方式
push(item)	按优先级顺序将值为item的新元素插入到队列中适当位置，无返回值	首先调用基础容器的 push_back 操作，然后调用堆排序算法 push_heap 对元素进行重新排列
pop()	删除队首元素（优先级最高的元素），无返回值	首先调用堆操作算法 pop_heap 删除堆顶元素，然后调用基础容器的 pop_back 操作
top()	返回队首元素的值	调用基础容器的 front 操作实现
empty()	若优先级队列为空，则返回 true ；否则，返回 false	调用基础容器的 empty 操作实现
size()	返回优先级队列中元素的数目	调用基础容器的 size 操作实现





适配器 与 容器

- 标准库中定义的容器适配器都是基于顺序容器建立的。
- 程序员在创建适配器对象时可以选择相应的基础容器类。
 - 1) stack适配器可以建立在vector、list或deque容器上。
 - 2) queue适配器只能建立在list或deque容器上。
 - 3) priority_queue适配器只能建立在vector或deque容器上。
- 如果创建适配器对象时不指定基础容器，则stack和queue默认采用deque实现，而priority_queue则默认采用vector实现。



适配器 与 容器 (Stack 为例)

定义于头文件 `<stack>`

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

`std::stack` 类是容器适配器，它给予程序员栈的功能——特别是 FILO（先进后出）数据结构。

模板形参

T - 存储的元素类型。

Container - 用于存储元素的底层容器类型。容器必须满足序列容器 (SequenceContainer) 的要求。

另外，它必须提供拥有通常语义的下列函数：

- `back()`
- `push_back()`
- `pop_back()`

标准容器 `std::vector`、`std::deque` 和 `std::list` 满足这些要求。若不为特定的 `stack` 类特化指定容器类，则使用标准容器 `std::deque`。



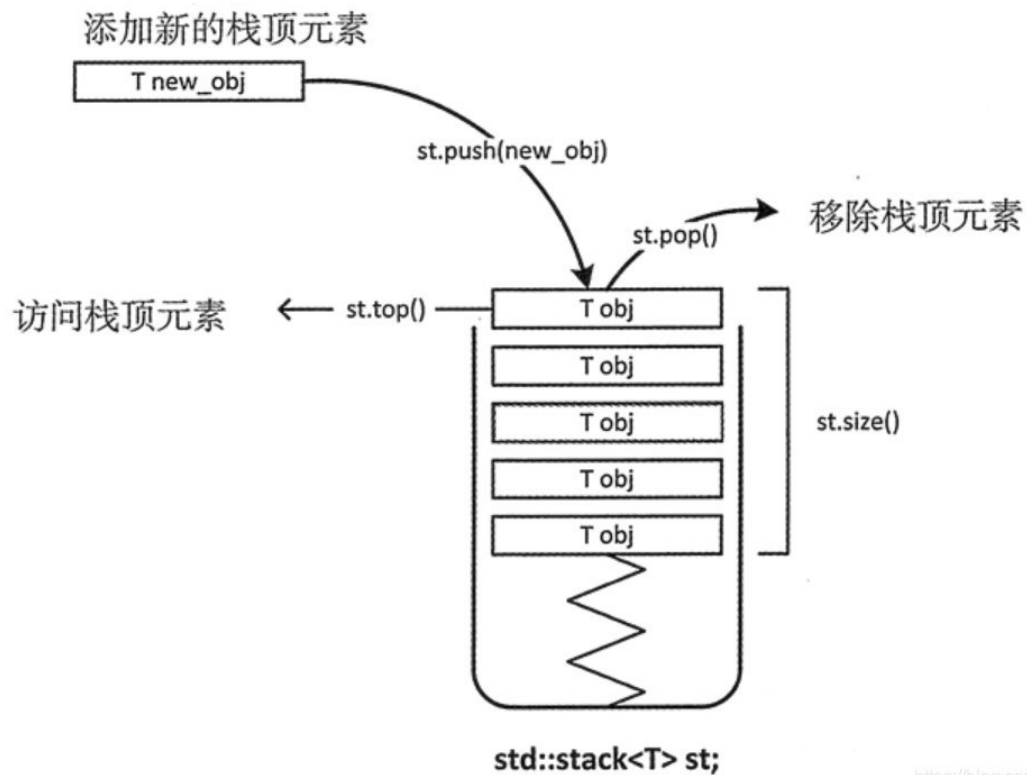
构造函数示例 (Stack 为例)

```
#include <stack>
#include <deque>
#include <iostream>

int main()
{
    std::stack<int> c1;
    c1.push(5);
    std::cout << c1.size() << '\n';

    std::stack<int> c2(c1);
    std::cout << c2.size() << '\n';

    std::deque<int> deq {3, 1, 4, 1, 5};
    std::stack<int> c3(deq);
    std::cout << c3.size() << '\n';
}
```





EXP: Stack

stack.cpp

```
// palindrome.cpp 功能： 判断给定文本是否回文（正读反读都一样的文本）
// 演示stack适配器的使用

bool palindrome(const string& text);

int main() {
    string text;
    while (true) {
        cout << "Enter the text(\"quit\" to end program):" << endl; // 输入文本
        getline(cin, text);
        if (text == "quit")
            break;
        if (palindrome(text)) // 是回文
            cout << "The text you typed is a palindrome." << endl;
        else // 不是回文
            cout << "The text you typed is NOT a palindrome." << endl;
    }

    return 0;
}
```



EXP: Stack

stack.cpp

```
bool palindrome(const string& text) {  
    stack<char, std::vector<int>>> cstack;           // 指定容器类型  
    size_t length = text.size();  
    size_t comparePos;  
    for (size_t i = 0; i < length/2; i++)           // 比较位置  
        cstack.push(text[i]);                     // 将文本text的前半部分压入栈cstack  
    // 将文本text的后半部分逐个字符与前半部分的对应字符进行比较  
    if (length % 2 == 0)                             // 设定比较起点  
        comparePos = length / 2;  
    else  
        comparePos = length / 2 + 1;  
    while (!cstack.empty()) {                       // 比较对应字符  
        if (text[comparePos] != cstack.top())       // 对应字符不相同  
            break;  
        cstack.pop();                               // 对应字符出栈  
        comparePos++;                               // 比较位置后移一个字符  
    }  
    if (cstack.empty()) return true;  
    else return false;  
}
```

目录

CONTENTS

01

STL是什么

02

容器

03

迭代器

04

算法

05

其他（选讲）



迭代器 – 类别 (1)

迭代器类别	功能	支持的操作	备 注
输入迭代器 (input iterator)	读	<code>const *</code> <code>const -></code>	输入迭代器的解引用 (<code>*</code>) 操作的结果只能作为右值 ^[1] 使用
输出迭代器 (output iterator)	写	<code>*</code> <code>-></code>	输出迭代器的解引用 (<code>*</code>) 操作的结果可以作为左值 ^[2] 使用

```
int main() {  
    // DEM01: read only iterator  
    const vector<int> iv {1,2,3,4,5,6,7};  
    auto it1 = iv.begin();  
    // *it1 = 3; // error: assignment of read-only location  
  
    set<int> is {1,2,1,3,1,4};  
    auto it2 = is.begin();  
    // *it2 = 5; // error: assignment of read-only location  
}
```

it-type.cpp



迭代器 – 类别 (2)

迭代器类别	功能	支持的操作	备 注
正向迭代器 (forward iterator)	读/写	++ ==,!=	向前访问下一个
双向迭代器 (bidirectional iterator)	读/写	++ , -- ==,!=	向前或向后
随机访问迭代器 (random access iterator)	读/写	++ , -- , + , - , += , -= ==,!= , < , <= , > , >= []	iter[n] 等价于 * (iter+n)

```
int main() {  
    // DEM02: forward, bidirection, random  
    forward_list<int> ifl {1,2,3,4,5,6,7};  
    list<int> ils {11,12,13,14,15,16,17};  
    auto it3 = ifl.begin();  
    it3++;  
    // it3--; // error: no 'operator--(int)' declared  
    auto it4 = ils.begin();  
    it4++; it4--;  
    // it4 += 7; // error: no match for 'operator+='  
    it1 += 7;  
}
```

it-type.cpp



迭代器 – 类别 (3)

容器类别	读/写	迭代器类别	备 注
array	R/W	random	
vector	R/W	random	
deque	R/W	random	
list	R/W	bidirectional	
set/multiset	R	bidirectional	
map/multimap	R	bidirectional	
forward_list	R/W	forward	



迭代器 – 遍历 与 for (C++11)

- 由每个类型的容器自行定义自己的迭代器。但我们仅关注类别特征
- 用 `auto` 关键字可以减少对迭代类型的依赖
 - 对于一般遍历，都可以写成右边范式代码

- 因此C++11以后，遍历容器引入新语法

```
for (const auto& x:container) {...}
```

- 其中，`container` 需要满足以下一个
 - 是 `array`
 - 或有 `begin()` 和 `end()` 成员函数的对象
 - 或 一个花括号初始化器列表
- 其中，`const` 和 引用 都是可选的

```
int main() {  
    //Demo3: iterating over the elements  
    auto _begin = ils.begin();  
    auto _end = ils.end();  
    for (auto it = _begin; it != _end; it++) {  
        auto x = *it;  
        cout << x << ",";  
    }  
    cout << endl;  
  
    for (auto x:ils) {  
        cout << x << ",";  
    }  
    cout << endl;  
}
```




迭代器 - 非法化（失效）

只读方法决不非法化迭代器或引用。修改容器内容的方法可能非法化迭代器和/或引用，总结于此表格。

类别	容器	插入后.....		擦除后.....		条件
		迭代器合法?	引用合法?	迭代器合法?	引用合法?	
顺序容器	array	N/A		N/A		
	vector	否		N/A		插入更改容量
		是		是		被修改元素前
		否		否		于被修改元素或其后
	deque	否	是 否	是，除了被擦除元素 否	修改首或尾元素 仅修改中部	
	list	是		是，除了被擦除元素		
	forward_list	是		是，除了被擦除元素		
关联容器	set multiset map multimap	超纲啦！		是，除了被擦除元素		
无序关联容器	unordered_set unordered_multiset	否	是	N/A		插入导致重哈希
	unordered_map unordered_multimap	是		是，除了被擦除元素		无重哈希

此处**插入**指代任何添加一或多个元素到容器的方法，而**擦除**指代任何从容器移除一或多个元素的方法。



迭代器 – 失效案例研究

it-type.cpp

```
int main()
{
    //EX: delete all 3 in vector.
    // 1.请指出程序中的缺陷, 并给出样例数据
    // 2.请改正该程序
    vector<int> ivv1 {1,2,3,4,5};
    for (auto it = ivv1.begin(); it != ivv1.end(); it++) {
        if (*it == 3) ivv1.erase(it);
    }
    for (auto x : ivv1) cout << x << ", ";
    cout << endl;
}
```

目录

CONTENTS

01

STL是什么

02

容器

03

迭代器

04

算法

05

其他（选讲）



可调用 (Callable) 类型 (选讲)

这里，“算法”特指头`<algorithm>`和`<numeric>`中定义的相关容器上的泛型算法。这些算法通常会涉及函数回调，在 C++ 中可调用的类型复杂了许多。

可调用 (*Callable*) 类型是可应用 *INVOKE* 和 *INVOKE<R>* 操作（例如用于 `std::function`、`std::bind` 和 `std::thread::thread`）的类型。

可调用类型大致包括以下几种情形：

- 普通函数（类的静态函数成员）及其指针
- 类的动态函数成员及其指针
- 函数对象，即重载 `operator()` 运算符类的对象
- 匿名函数，即 **Lambda表达式**

为了减少复杂性，这里仅给出一些案例帮助大家理解和实战应用



可调用 - 普通函数回调 (选讲)

函数回调-普通函数.cpp

```
#include<iostream>

typedef int (*FP)(int,int);

int Add(int lhs,int rhs) {
    return lhs+rhs;
}

int Mul(int lhs,int rhs) {
    return lhs*rhs;
}

void callback_client(FP f, int a, int b) {
    std::cout << f(a,b) << std::endl;
}

int main() {
    callback_client(Add, 2, 3);
    callback_client(Mul, 2, 3);
}
```



可调用 - 类成员函数回调 (选讲)

函数回调-类成员函数.cpp

```
typedef int (*FP)(int,int);

class Foo{
public:
    int Add(int lhs,int rhs) {...}
    static int Mul(int lhs,int rhs) {...}
};

void callback_client(FP f, int a, int b) {
    std::cout << f(a,b) << std::endl;
}

void callback_client(int (Foo::*f)(int,int), Foo &foo, int a, int b) {
    std::cout << (foo.*f)(a,b) + 1000 << std::endl;
}

int main() {
    Foo foo;
    callback_client(foo.Mul, 2, 3);
    callback_client(Foo::Mul, 2, 3);
    //callback_client(foo.Add, 2, 3); //error
    //callback_client(Foo::Add, 2, 3); //error
    callback_client(foo.Add, foo, 2, 3);
    callback_client(Foo::Add, foo, 2, 3);
    // bar 类也有 Add 成员函数 ... 怎么办? 继续重载?
}
```



可调用 - std::bind 和 std::function (选讲)

std::bind 可调用对象的绑定器 (包装器)

在标头 `<functional>` 定义

```
template< class F, class... Args > (C++11 起)
/* 未指定 */ bind( F&& f, Args&&... args ); (C++20 前)
(1)
```

参数

- f - 可调用 (Callable) 对象 (函数对象、函数指针、函数引用、成员函数指针或数据成员指针)
- args - 要绑定的参数列表, 未绑定参数会被命名空间 std::placeholders 的占位符 _1, _2, _3... 替换

返回值

- 某个未指定类型 T 的函数对象 g, 其中 std::is_bind_expression<T>::value 是 true。

类模板 std::function 是通用多态函数类型包装器

在标头 `<functional>` 定义

```
template< class > (C++11 起)
class function; /* 未定义 */

template< class R, class... Args > (C++11 起)
class function<R(Args...)>;
```



可调用 - std::bind 和 std::function (选讲)

函数回调-std-bind.cpp

```
#include<iostream>
#include<functional>

typedef std::function<int (int,int)> FP; //这个类型的可调用对象

class Foo{
public:
    int Add(int lhs,int rhs) {...}
    static int Mul(int lhs,int rhs) {...}
    int operator()(int lhs,int rhs) { return this->Add(lhs,rhs);}
};

void callback_client(FP f, int a, int b) {
    std::cout << f(a,b) << std::endl;
}

int main() {
    using namespace std::placeholders; // for _1, _2, _3...
    Foo foo;
    callback_client(foo.Mul, 2, 3);
    callback_client(Foo::Mul, 2, 3);

    auto f = std::bind(foo.Add,&foo,_1,_2); //包装成 FP 类型的函数对象
    callback_client(f, 2, 3);
    callback_client(foo, 2, 3); //函数对象, 更方便
}
```




可调用 - lambda表达式 (匿名函数) (选讲)

能够捕获作用域中的变量的无名函数对象。

语法

[捕获列表](参数列表) -> 返回类型 { 函数体 }

结果

- 某个未指定类型 T 的函数对象 g , 类型 `function<R(Args...)>`

```
#include<iostream>

int main() {
    auto f1 = []{return "hello lambda!";};
    std::cout << f1() << std::endl;

    int i = 3;
    auto f2 = [&i](int a){i++; return a+i;};
    std::cout << i << "-" << f2(5) << std::endl;
}
```

callable对象-lambda.cpp



算法

此处说的“算法”指<algorithm>和<numeric>定义的各种容器上的泛型算法：
全部可用算法：cppreference
这里介绍几个使用实例。

sort.cpp

```
// sort.cpp, compile with -std=c++11
int main() {
    using namespace std;

    //vector vec = { 13219, 1203, 3213, 4132 }; // c++20 支持类模板推导
    vector<int> vec = { 13219, 1203, 3213, 4132 };
    sort(vec.begin(), vec.end());

    for (auto i : vec) {
        std::cout << i << " ";
    }
}
```

这样是升序，如何降序排序？



算法

使用<algorithm>时常常遇到类似问题，一般传入函数即可实现新的功能。

sort2.cpp

```
// sort2.cpp, compile with -std=c++11
int main() {
    using namespace std;

    vector<int> vec = { 13219, 1203, 3213, 4132 };
    sort(vec.begin(), vec.end(), [](int a, int b) { return a > b; });

    for (auto i : vec) {
        std::cout << i << " ";
    }
}
```

这个是函数指针？

这个是 lambda 函数！



算法

<algorithm>中有一个count_if函数，如果想要统计一个vector中大于15和大于20的数的数量，可以这么写：

count-if.cpp

```
// count_if.cpp compile with -std=c++11
int main() {
    using namespace std;

    vector<int> vec = { 1, 10, 20, 15, 30 };

    auto greater_then_20 =
        count_if(vec.begin(), vec.end(), [](int i) { return i > 20; });
    auto greater_then_15 =
        count_if(vec.begin(), vec.end(), [](int i) { return i > 15; });

    cout << greater_then_20 << "\n"
         << greater_then_15 << "\n";

}
```



算法

<algorithm> 提供了 accumulate:

```
// accumulate.cpp, compile with -std=c++11/20
int main() {
    using namespace std;
    //vector v = { 1, 2, 3, 4 };
    vector<int> v = { 1, 2, 3, 4 };
    //auto res = accumulate(v.begin(), v.end(), 1, [](auto i, auto j) { return i
    * j; });
    auto res = accumulate(v.begin(), v.end(), 1, [](decltype(v[0]) i,
    decltype(v[0]) j) { return i * j; });
    cout << res;
}
```

accumulate.cpp

累加初值

函数如 std::multiplies

这段代码将v中的所有整数相乘，并存入res中。

目录

CONTENTS

01

STL是什么

02

容器

03

迭代器

04

算法

05

其他 (选讲)



STL编译时发生了什么？

由于C++模板的限制，STL以头文件方式分发。

这也就是说，编译STL时首先会将STL里的所有内容都复制粘贴到源文件中

```
$ time g++ bits.cpp
```

```
real    0m1.005s
```

```
user    0m0.834s
```

```
sys     0m0.041s
```

编译毫无内容的bits.cpp，编译器用了惊人的0.834秒

```
// bits.cpp  
#include <bits/stdc++.h>  
  
int main() {}
```



STL编译时发生了什么？

```
$ g++ -E bits.cpp | wc -l  
105803
```

将所有C++标准库包含的后果是：

预处理后得到105803行的源文件！



建议与展望

建议：绝对不要在工程项目中使用bits/stdc++.h，只引入需要的头文件。

C++20引入了现代编程语言一般都具有的module系统，可以解决头文件带来的编译速度问题。

但不要过于乐观，C++标准库在C++20中仍然以头文件分发。目前只有微软实现了C++标准库的module化，msvc16.8以上版本支持以下写法：

```
import std.core;

using namespace std;

int main()
{
    cout << "hello, world";
}
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组